System summary up to now

The HRMS is built as a role-based web application where authentication, employee management, department management, requests, and admin/HR operations are connected through a structured database. The system is designed so that **User** handles login and access control, while HR stores **Employee** related profile data. This separation makes the system cleaner, safer, and easier to manage.

Core working functions

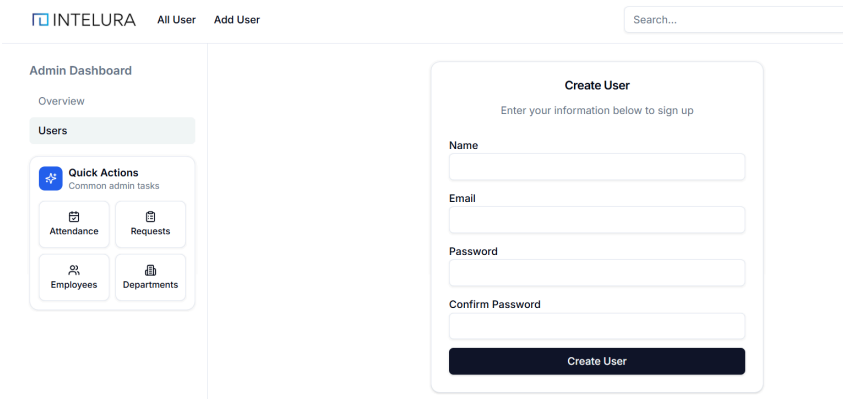
At this stage, the main working areas are

Authentication and access control

- Users can sign in through the authentication system.
- Access is controlled by role.
- Admin and HR users are given elevated dashboard access.
- Regular users do not see restricted admin features.

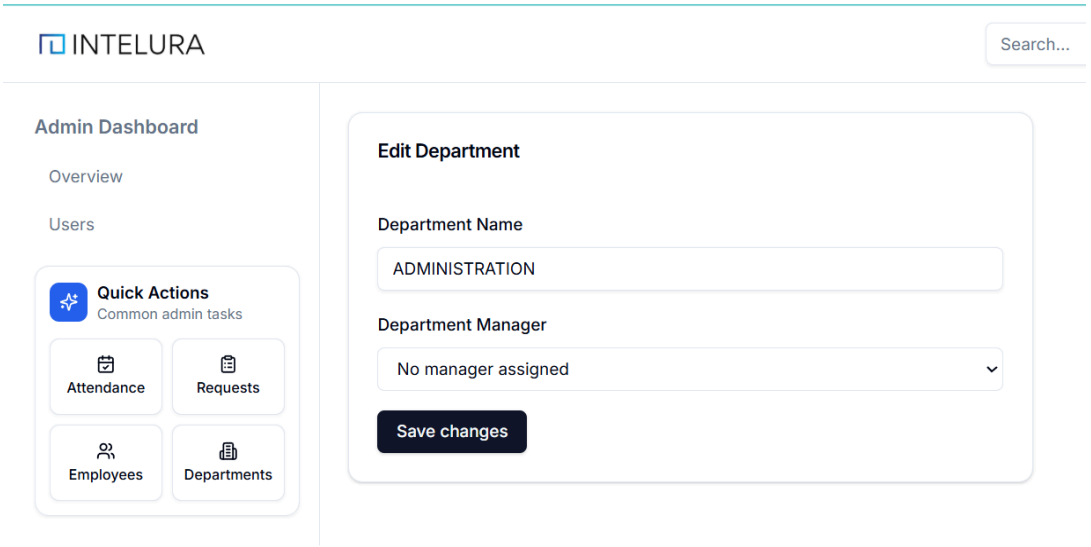
User and employee structure

- A user account is created first.
- Employee records depend on the user record.
- Employee data is linked to the user so the system knows who the logged-in person is and what HR data belongs to them.



Department management

- Departments can be created, edited, and deleted from the admin/HR side.
- Manager selection is linked to department setup.
- Department data is now intended to come from the database instead of hardcoded enums.
- We can create/edit department by Typing but manager selection data is retrieved by the Employee table.



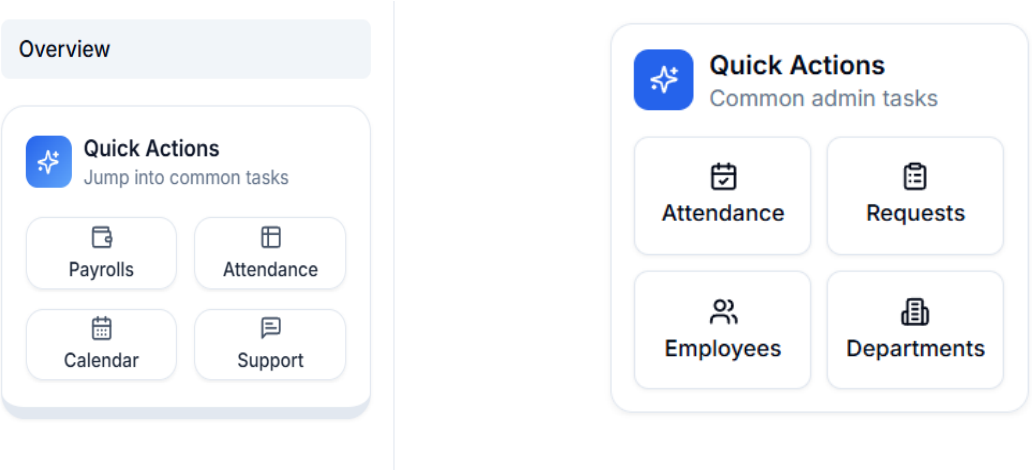
Request forms

- UI has been prepared for request types such as leave, claims and support is the general requests.
- Data is being retrieved dynamically.
- Manager-related field logic is part of the submission flow.

  [Access Request forms]

Quick actions and dashboard

- Quick Actions are available on the left side of the dashboard.
- Admin has its own action set.
- Different roles can see different actions depending on permissions.



Data flow

The system follows this general flow:

1. User creation

A person is first created as a **User** in the database.
This record contains things needed for login and access, such as:

- name
- email
- password/authentication details
- role

2. Employee creation

After the user exists, an **Employee** record is linked to that user.
The employee record stores HR-related information such as:

- department
- position
- branch
- manager
- other employment details

So the relationship is:

User → Employee

This means:

- a user account provides system access
- the employee profile provides business and HR identity inside the company

3. Login and session use

When the user signs in:

- authentication checks the user credentials
- a session/token is created
- the system reads the user's role
- based on that role, the UI and routes are controlled

4. Feature access

After login, the system decides:

- which dashboard to show

- which menu items to show
- which actions are allowed
- which pages or APIs should be restricted

How users are added

Users are typically added into the database first.
That user then becomes the base identity for:

- authentication
- authorization
- role checking

Once the user exists, the employee details can be connected to that same user.This structure is important because not every system-level user must hold all HR data directly in the same table. It keeps login concerns and employment concerns separate.

How employees depend on users

Employees depend on users because the employee record is tied to the user record.

That means:

- no proper employee profile should exist without a related user
- when someone logs in, the system can map the logged-in user to their employee record
- role and login status come from **User**
- department, manager, and HR information come from **Employee**

So in simple terms:

- **User = system identity**
- **Employee = company identity**

Authentication

Authentication is handled through the sign-in system using the application auth setup.Current authentication logic includes:

- credential-based login
- role checking
- session handling
- protected access based on role

From your recent work, Admin access was extended to HR as well, so HR users can now access the admin-type panel features where permitted.

HR/Admin functions and options

HR/Admin currently has or is being given access to functions like:

- department management
  - create department
  - edit department
  - delete department
  - assign/select manager for department

- employee-related administration
  - manage employee-linked data
  - assign department and related details

- dashboard/admin access
  - quick actions
  - admin-side navigation and modules

- request oversight
  - request forms and related workflow setup
  - manager-dependent approval flow preparation

What is restricted for normal users

Regular users are restricted from admin and HR management functions.

Restricted areas include:

- department creation/edit/delete
- admin dashboard tools
- HR/admin quick actions
- higher-level management settings
- protected routes/pages that require role-based authorization

In practice, users only see what their role allows.
So the UI and backend should both prevent unauthorized access.

Database and how it is connected

The system uses **PostgreSQL** as the database and **Prisma ORM** as the connector between the Next.js application and the database.

Connection structure

- **PostgreSQL (NEON)** stores all system data
- **Prisma schema** defines models and relations
- **Prisma Client** is used inside the app to read and write data

- the app connects to the database through the `DATABASE_URL` environment variable

Main connection flow

- frontend form sends data
- server action or API route receives it
- Prisma runs create/update/find queries
- PostgreSQL stores or returns the data
- UI updates with the returned data

Main database model idea

The important design approach is:

- **User** handles login, role, and account identity
- **Employee** handles HR profile details
- **Department** stores department records
- other modules connect around these, such as requests and management features

This gives a normalized structure and makes the system easier to scale.